COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

09

VOLUME NINE

PACKAGE 10 / GRADUATE SPECIALIZATION

Advanced rendering and reconstruction

Standalone teaching module; use the workbook and separate answer key

MODULE 43 / FOUR LESSONS

Edition: September 28, 2026

Original lesson sequence preserved. Self-study materials and runnable educational references; not an accredited degree or completed learner submission.

Contents

Module 43 — Advanced rendering and reconstruction 3

Lesson 43.1 — Monte Carlo light transport and multiple importance sampling 4

Lesson 43.2 — Real-time ray tracing, dynamic acceleration and hybrid pipelines 7

Lesson 43.3 — Spatiotemporal reuse, reservoirs and denoising 9

Lesson 43.4 — Differentiable and neural rendering under engineering constraints 11

Module 43 project: bounded rendering reproduction 13

Primary reading directory 14

Module 43 — Advanced rendering and reconstruction

Prerequisites: Modules 09-10, 21-23 and 41-42.

Original project brief: Reproduce a bounded rendering result and add one justified extension. Derive the estimator or reconstruction objective, compare equal-quality cost, include temporal failure cases and preserve source scenes and evaluation scripts.

Core reference IDs: R23, R44, R47. Full entries are in the reading directory.

- 43.1 — Monte Carlo light transport and multiple importance sampling
- 43.2 — Real-time ray tracing, dynamic acceleration and hybrid pipelines
- 43.3 — Spatiotemporal reuse, reservoirs and denoising
- 43.4 — Differentiable and neural rendering under engineering constraints

Lesson 43.1 — Monte Carlo light transport and multiple importance sampling

Learning outcome: Derive estimators with correct densities and measures.

Planning: 30-46 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

The rendering equation expresses outgoing radiance as emission plus an integral of incident radiance, scattering and geometry factors. Monte Carlo estimation samples an integral: for samples from density p, average f(x)/p(x) where p is positive over the relevant nonzero support. Correct units and probability measures matter; area and solid-angle densities are not interchangeable without a Jacobian. Importance sampling aims to reduce variance by matching useful structure. Multiple importance sampling combines strategies with weights that sum appropriately under the estimator. The balance heuristic uses relative weighted densities, but sample counts and support must be included correctly. Russian roulette can terminate paths without bias when surviving contributions receive the proper probability compensation. Finite-variance independent sampling gives the familiar inverse-square-root standard-error scaling; do not assume it for every estimator.

An estimator is a mathematical object, not a bag of samples

For I=integral f(x) dx and samples X drawn from density p with respect to that same measure, E[f(X)/p(X)]=I when support and integrability conditions hold. A density is not a probability at a point; its units depend on the domain. In light transport, scattering terms, cosine factors, emitted radiance and visibility combine into an integrand. Losing one factor can produce plausible images while estimating the wrong quantity.

For a sampled light surface point, solid angle and surface area satisfy d_omega=abs(cos_light)/distance^2 * d_area. Therefore p_omega=p_area*distance^2/abs(cos_light). Light-selection probability is another factor: choosing one of K lights uniformly multiplies the conditional density by 1/K. Grazing directions can make p_omega large; skipping the Jacobian is not a harmless approximation. [R23, S07]

Derive multiple importance sampling with counts intact

Let technique i contribute n_i samples from p_i. An MIS estimator is sum_i (1/n_i) sum_j w_i(X_ij) f(X_ij)/p_i(X_ij). Weights must sum to one on the covered support and vanish where the associated technique cannot sample. For the balance heuristic, w_i(x)=n_i p_i(x)/sum_k n_k p_k(x). The count terms matter when sample allocation differs. This is not the same formula as drawing one random technique and forgetting its selection probability. [S07]

Use a scalar integral as an oracle before debugging full images. For f(x)=x^2 on [0,1], I=1/3. The reference combines uniform p1=1 and p2=2x, drawing p2 samples as sqrt(U). Each strategy is weighted against both densities. Repeated seeds reveal estimation noise; one seed near 1/3 proves little by itself. Support errors and density errors should be tested deliberately.

Connect the derivation to an executable path tracer

The included CPU reference has diffuse spheres, a black environment, explicit surface-light sampling, BSDF sampling, a BVH, emission handling and balance weights. BSDF throughput for cosine-sampled Lambertian reflection simplifies to albedo. When a BSDF path reaches an emitter, the light PDF is reconstructed for the same direction so the competing contributions can be weighted consistently. [S08]

Russian roulette uses a survival probability q>0 and divides surviving throughput by q. That preserves expectation for the remaining estimator, but it does not remove the reference's separate finite-depth truncation: paths longer than six bounces are omitted. State that bias. Fixed origin offsets also depend on scene scale. Linear radiance values are saved to JSON; the PPM is a tone-mapped preview and must not be used for numerical error claims. The reference is a correctness microscope, not a real-time production renderer.

Worked engineering example

For one sample from each technique, p1=2 and p2=6 give w1=2/(2+6)=0.25. With three samples from technique 1 and one from technique 2, w1=(3*2)/(3*2+1*6)=0.5. *For a Lambertian surface with albedo rho under constant incoming radiance L, cosine sampling uses p=cos(theta)/pi. The contribution [rho/pi * L * cos(theta)]/p is rho*L for every nongrazing sample. With rho=0.8 and L=2, the exact reflected radiance is 1.6. This analytically checkable case catches missing pi and cosine factors.

Original worked check

With one sample from each of two strategies and densities p1=2 and p2=6 at a point, the balance weight for strategy 1 is 2/(2+6)=0.25.

Practical assignment

Implement a path tracer with explicit light sampling and MIS. Pass: derive the estimator, verify a simple known integral, compare variance at equal cost and test emission and grazing-angle edge cases.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_43_1.py`.

Reference starting point: code/gradlab/rendering.py; code/run_experiments.py render

Assigned study

Original assignment: R23: Monte Carlo Integration, Light Transport and sampling strategies. Work the associated mathematical exercises, not only the implementation.

Focused expansion: R23: Monte Carlo Integration and Light Transport I; S07: Improving Efficiency, especially MIS; S08: A Better Path Tracer. Work through the PDF measures explicitly before porting any code.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What is the balance weight for strategy 1 in the example?
2. Why is $p(x)>0$ on relevant support important?
3. What sample-count increase ideally halves standard error for independent finite-variance samples?

Lesson 43.2 — Real-time ray tracing, dynamic acceleration and hybrid pipelines

Learning outcome: Budget traversal, building and shading as one system.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Ray-tracing acceleration structures trade build cost, memory and query cost. Dynamic scenes may update or rebuild structures; the best policy depends on movement, geometry and traversal degradation. A refit can be cheaper initially but leave poor bounds after large changes. Hardware-assisted traversal still needs correct synchronization and resource lifetime. Hybrid renderers combine rasterized visibility with ray-traced shadows, reflections or indirect lighting. Each path needs a consistent material and coordinate model. Screen-space or cached approximations require fallbacks where information is absent. Measure acceleration build/update, tracing, shading, denoising and synchronization separately, then report total frame impact. Compare equivalent scene motion and image quality rather than static screenshots alone.

The best acceleration structure depends on its lifetime

A hierarchy groups primitives into bounds so a ray can reject many primitives at once. A tighter or more carefully partitioned tree may cost more to build yet reduce traversal. Refit preserves topology and updates bounds after motion; rebuilding changes the partition. Evaluate total build/update plus query work over the reuse interval, not just the shortest traversal time.

For a structure rebuilt once and reused for F frames, per-frame amortized cost is build/F+trace, assuming comparable frames. That assumption can fail under motion: a cheap refit can leave broad overlapping bounds that worsen traversal. A policy should consider predicted or observed query cost, memory and update frequency. The reference uses a median-split sphere BVH rather than a sophisticated surface-area-heuristic builder, giving the learner room for a meaningful extension.

Correctness includes degenerate rays and changing geometry

A slab test intersects a ray interval against each axis-aligned box. For a zero direction component, check whether the origin lies inside that slab instead of dividing by zero. Intersections must use a consistent t interval and handle rays starting inside bounds. Compare the accelerated result against brute force for randomized rays, then test tangency, parallel rays and close surfaces explicitly.

The supplied refit assumes the same primitive identities and topology remain present. Adding or removing primitives is not covered by that operation. A dynamic-scene test should move geometry enough to change overlap and query coherence, rather than only translating the whole scene together. Track primitive tests or node visits as explanatory metrics; elapsed time alone may hide why a refit loses.

Hybrid pipelines share resources and meaning

A rasterized primary view can feed ray-traced shadows or reflections, but a ray effect must still agree about materials, coordinate frames and visibility. Screen-space information cannot describe an offscreen occluder, so a fallback must state what happens when information is missing. Do not quietly compare a screen-space approximation with full-scene rays as if they had identical quality.

Hardware acceleration adds build resources, scratch memory, device addresses, synchronization and object lifetimes. Keep source geometry and scratch allocations valid for the durations required

by the API, and establish a build-write to trace-read dependency. The existence of a ray-tracing feature does not establish that every update mode or geometry change is valid. The CPU model validates intersection logic only; it does not validate those hardware lifetime requirements. [S09]

Worked engineering example

Policy A takes 1 ms to refit and 5 ms to trace each frame. Policy B takes 5 ms to rebuild and 2 ms to trace. If every frame uses a fresh structure, A costs 6 ms and B costs 7 ms, so rebuilding loses. If a static structure is built once and reused for F frames, total times are 1+5F and 5+2F. B wins when 5+2F<1+5F, meaning F>4/3, so it wins from two identical-use frames onward under that simplified model. In a moving scene, the fixed 5 ms trace assumption must be retested.

Original worked check

A faster traversal variant that adds 4 ms of build work to save 1 ms of tracing is a regression for a scene rebuilt every frame, but might help a scene reused for many frames.

Practical assignment

Add one ray-traced effect to the engine or build a software hybrid prototype. Pass: dynamic-scene tests include total update cost, missing-information fallbacks and resource synchronization evidence.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_43_2.py`.

Reference starting point: code/gradlab/rendering.py: BVH, Sphere, path_radiance

Assigned study

Original assignment: R23: intersection acceleration and light transport; R21, R24 or R25: supported ray-tracing interfaces for an optional hardware route.

Focused expansion: R23: Primitives and Intersection Acceleration; S09: Vulkan acceleration-structure specification for an optional hardware route. Follow the exact API version used by your project for build/update flags and synchronization.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can a cheap BVH refit become a poor long-term choice?
2. Is saving 1 ms of tracing worth 4 ms of per-frame rebuild cost?
3. Why compare animated scenes?

Lesson 43.3 — Spatiotemporal reuse, reservoirs and denoising

Learning outcome: Evaluate variance reduction together with bias, history validity and motion.

Planning: 30-46 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Temporal and spatial reuse exploit related samples across frames or nearby locations. Reusing data changes the sampling and correlation structure, so the estimator and weights must be derived rather than improvised. Reservoir sampling can retain a compact representative sample under a defined weighted selection rule; its rendering use requires additional treatment of target distributions and reuse. Denoising reconstructs a signal from noisy observations using assumptions about spatial, temporal or learned structure. It can suppress noise while introducing bias, blur or ghosting. Visibility changes and incorrect motion vectors invalidate history. Distinguish estimator bias from reconstruction bias and assess moving sequences at equal total cost. The original 2020 ReSTIR paper is a historical reproduction option, not a claim to represent the latest technique.

Prove the weighted reservoir before reusing it

For nonnegative weights w1,...,wM, keep a running sum W and replace the current selection with item i with probability wi/W after adding its weight. Inductively, an earlier item's selection probability becomes [wj/W_old]*[W_old/W_new]=wj/W_new. The newly arrived item has probability wi/W_new. Thus the final item is selected proportionally to its weight. Reject negative and nonfinite weights. If all weights are zero, no representative is selected.

This proves a selection rule, not a rendering estimator. The reference's frequency test checks weights (1,2,7), expecting frequencies near (0.1,0.2,0.7). Statistical tolerance is declared for a fixed seed and trial count; it is not a proof of every distributional property. The all-zero case and overflow behavior are just as important as the attractive positive-weight case.

Selection weights and estimator normalization are different

Consider M independent candidates Xi~q, target surrogate p_hat positive wherever the desired contribution f is nonzero, and weights wi=p_hat(Xi)/q(Xi). After weighted selection of Y, the estimator [W/(M*p_hat(Y))]*f(Y) has conditional expectation (1/M) sum_i f(Xi)/q(Xi). Taking expectation over candidates gives the desired integral when support and integrability conditions hold. This derivation explains why keeping only Y without W and M loses essential information.

Spatial or temporal reservoir reuse changes the candidate-generation structure, the target and the correlations. Reusing a neighbor's representative as though it were a fresh draw from your proposal does not inherit the derivation above. The selected research method supplies additional normalization and validity conditions; derive those for the exact bounded variant being reproduced. Do not call a weighted reservoir implementation "ReSTIR" by itself. [R44]

History acceptance is a geometric and statistical decision

A temporal filter blends current c with reprojected history h: output=alpha*c+(1-alpha)*h for valid correspondence, otherwise output=c. The same pixel address is not correspondence when objects or the camera move. Depth, surface identity, normals, motion vectors and visibility changes can provide rejection evidence, but thresholds themselves create tradeoffs.

Noise reduction can introduce blur, lag, ghosting or estimator/reconstruction bias. Test disocclusion, a camera cut, moving highlights and changing illumination. Separate raw estimator error from reconstruction error by evaluating both before and after filtering. Sequence-level metrics and selected temporal traces expose failures that a favorable still frame hides. Total cost includes history storage, motion processing, rejection and reconstruction, not only rays saved.

Worked engineering example

Suppose an old pixel is 1 and a newly revealed background is 0. With alpha=0.2 and no rejection, the filter outputs 0.8, then 0.64, then 0.512 on successive zero observations. After ten frames the remnant is 0.8^10, approximately 0.1074. The noise-free input has acquired a visible error entirely from invalid history. Correct rejection outputs 0 immediately in this simplified case. A denoiser can therefore lower static-frame noise while becoming less accurate at a visibility transition.

Original worked check

If a moving object reveals a background, old foreground samples are not valid background evidence merely because they occupy the same pixel coordinate. History acceptance must account for correspondence and visibility.

Practical assignment

Reproduce a small weighted-reservoir example or temporal denoiser, then evaluate on motion and disocclusion sequences. Pass: the selection/weight rule is derived, bias limitations are stated and results include sequence-level failure cases.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_43_3.py`.

Reference starting point: code/gradlab/rendering.py: Reservoir, temporal_filter

Assigned study

Original assignment: R44: original ReSTIR paper and its estimator assumptions; R23: sampling theory; read only the selected bounded method before expanding scope.

Focused expansion: R44: Bitterli et al., ReSTIR (2020), estimator and reuse assumptions; R23: sampling theory. The executable Reservoir class is only weighted stream selection, not the complete paper algorithm.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why cannot reused samples always be treated as new independent samples?
2. Why is same-pixel history not enough to establish correspondence?
3. What should be compared in a denoised rendering benchmark?

Lesson 43.4 — Differentiable and neural rendering under engineering constraints

Learning outcome: Test learned or gradient-based methods against explicit baselines.

Planning: 28-44 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Differentiable rendering studies how outputs change with scene or model parameters. Visibility discontinuities, stochastic sampling and approximations complicate gradients. A useful derivative must match the actual objective and parameterization; automatic differentiation alone does not guarantee the right physical or statistical estimator. Validate on small scenes with finite differences and known cases. Neural rendering or learned reconstruction can trade preprocessing, training and memory for runtime quality or speed. Training data coverage, scene specificity, temporal stability and inference hardware constrain deployment. Compare a non-learned baseline and include training or preprocessing cost where the claim requires it. Separate interpolation within familiar conditions from generalization to new scenes, lighting or motion.

Differentiate the actual inverse problem

Let observations y_i be generated by a rendering model R_i(theta). A squared-error objective is 0.5 sum_i (R_i(theta)-y_i)^2, with gradient sum_i (R_i-y_i)*dR_i/dtheta. Differentiating an implementation produces derivatives of that implementation's computational path. It does not automatically account for omitted visibility changes, discontinuous sampling choices or a mismatch between the modeled camera and the data. [S10]

Start with a smooth, identifiable subproblem: known lighting and scalar Lambertian albedo. Write R_i=rho*b_i with b_i=E*max(cos_i,0)/pi. The unconstrained optimum is rho=sum(b_i*y_i)/sum(b_i^2), projected to [0,1] for the physical albedo bound. The denominator measures sensitivity; all-dark observations provide no information about rho. The executable reference rejects that unidentifiable case instead of inventing an estimate.

Gradient checks need a noise and discontinuity policy

For deterministic smooth models, compare analytic derivatives with centered differences over a range of step sizes. For stochastic estimators, independent random samples at theta+h and theta-h can make the difference dominated by noise. Common random numbers can reduce variance when used correctly, but do not make a discontinuous or biased derivative estimator mathematically valid.

A silhouette crossing can change which surface is visible abruptly. A local derivative computed while holding the sampled path fixed may miss a boundary contribution to the desired derivative. State whether the experiment excludes visibility changes, uses an approximation or employs a method specifically designed for them. The small albedo fit deliberately keeps visibility and illumination fixed; it is a baseline, not a full differentiable renderer.

A learned method carries an additional engineering bill

For a neural route, define training inputs, targets, architecture, loss, optimizer, random seeds and stopping criterion. Hold out entire scenes when correlated frames would leak content. Compare with a non-learned baseline such as a simple temporal/spatial filter or analytic fit, not only with a deliberately undertrained network. Report training/preprocessing cost, inference time, model weights, activation memory and temporal behavior. [R47]

A preparation cost C amortized over N uses contributes C/N per use. Training can be acceptable for a repeatedly reused level yet impractical for user-generated content changed every minute. Distribution shifts include new materials, lighting, camera motion or hardware precision. Keep the performance claim limited to the tested shifts, and preserve a fallback that has a known numerical and latency contract.

Worked engineering example

Let E=1, cosine values (0.2,0.5,1) and true rho=0.7. Then b=cosine/pi and y=0.7*b. The numerator of the least-squares formula is 0.7 sum(b^2), so the fitted albedo is exactly 0.7 in exact arithmetic. If you instead try to estimate both rho and unknown E from these data, only their product is observable: doubling E and halving rho leaves predictions unchanged while both remain valid. More optimizer iterations cannot remove that identifiability ambiguity; it needs a known light, a prior or additional observations.

Original worked check

A reconstruction network tested on frames from its own training sequence may perform well without generalizing to a new level. Holding out entire scenes tests a stronger claim.

Practical assignment

Implement a small inverse-rendering optimization or learned reconstruction experiment. Pass: gradient or generalization checks, a non-learned baseline, compute/memory accounting and a distribution-shift failure analysis are included.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_43_4.py`.

Reference starting point: code/gradlab/rendering.py: inverse_albedo; code/gradlab/numerical.py: finite_gradient

Assigned study

Original assignment: R47: automatic differentiation, optimization and generalization; R23: light transport and sampling foundations; select one original method with Module 41's paper-review protocol.

Focused expansion: R47: automatic differentiation and generalization; S10: Mitsuba gradient-based optimization tutorial; R23: the light-transport model you differentiate. Choose an original neural method only for the optional learned route.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Does automatic differentiation guarantee the desired gradient estimator?
2. Why hold out entire scenes for some rendering evaluations?
3. When should training cost be included?

Module 43 project: bounded rendering reproduction

Select one result from an original rendering method and reconstruct its estimator or objective. A diffuse MIS renderer, a specific reservoir-reuse variant, a temporal reconstruction method or a smooth inverse-rendering problem can provide the setting, but a primitive alone is not a complete reproduction of a larger paper.

Preserve scene geometry, camera paths, lights, materials, seeds and linear reference images. Measure error against an adequately converged or analytically known reference, including reference uncertainty. Compare at equal total cost and at a declared quality threshold where feasible. Add motion/disocclusion or a justified adverse setting. Separate sampling error from reconstruction bias and include build, shading, history and preprocessing costs appropriate to the claim.

Required artifacts: paper-to-code map, estimator derivation, source scenes, raw linear outputs, error/time analysis, temporal failure evidence, one extension and limitations. Estimator/objective correctness 35 points, baseline/equal-quality comparison 25, temporal/adverse cases 20, artifact/review 20. Suggested pass: 80/100 and no unresolved density-measure or data-leakage error.

Primary reading directory

R01-R55 retain the original program IDs. S01-S17 are focused additions for this expansion. Citations such as [S05] refer to the entries below; equations and worked cases not attributed to another source are original teaching derivations. External resources retain their own rights and are linked rather than redistributed. Moving documentation is not a promise of compatibility with an untested installed version. Record exact versions for your own artifact.

R21 — Vulkan tutorial

Khronos Group

https://docs.vulkan.org/tutorial/latest/00_Introduction.html

Explicit graphics API study; feature-check the device before using the moving latest tutorial.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R23 — Physically Based Rendering, fourth edition

Matt Pharr, Wenzel Jakob, Greg Humphreys

https://www.pbr-book.org/4ed/contents

Rendering mathematics and implementation; use chapter titles in lesson assignments.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R24 — Direct3D 12 graphics

Microsoft

https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-graphics

Windows graphics backend reference.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R25 — Metal developer resources

Apple

https://developer.apple.com/metal/

Apple-platform graphics and compute reference.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R44 — Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

Benedikt Bitterli and coauthors

https://research.nvidia.com/publication/2020-07_spatiotemporal-reservoir-resampling-real-time-ray-tracing-dynamic-direct

Original ReSTIR research, 2020; example reproduction target, not a claim to be the latest method.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R47 — Dive into Deep Learning

Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola

https://d2l.ai/

Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

S07 — PBRT 4e: Improving Efficiency

Matt Pharr, Wenzel Jakob and Greg Humphreys

https://www.pbr-book.org/4ed/Monte_Carlo_Integration/Improving_Efficiency

Importance sampling, MIS and roulette.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S08 — PBRT 4e: A Better Path Tracer

Matt Pharr, Wenzel Jakob and Greg Humphreys

https://www.pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/A_Better_Path_Tracer

Light sampling and emitter-hit weighting; compare against the bounded educational implementation.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S09 — Vulkan specification: Acceleration Structures

Khronos Group

https://docs.vulkan.org/spec/latest/chapters/accelstructures.html

Capability, update and resource contracts. Moving specification, not a tested API backend.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S10 — Gradient-based optimization tutorial

Mitsuba project

https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering/gradient_based_opt.html

Primary implementation documentation for inverse rendering; no Mitsuba installation required by the portable references.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.